Atelier de professionnalisation

Partie 5.2 : Refactoring

Application mobile de gestion des notes de frais — GSB

Introduction

Votre application fonctionne, mais le code est dupliqué : trois fichiers lisent AsyncStorage, construisent l'URL et font leur propre fetch. Dans cet atelier, vous allez réorganiser le code sans changer ce que l'utilisateur voit.

Objectifs :

- le formulaire FraisForm ne fait plus que la saisie ;
- les écrans appellent l'API via un service unique (fraisService.js) ;
- le token vient toujours de useAuth().

Le principe : l'écran décide, le composant affiche

Le couple DashboardScreen / FraisCard suit déjà la bonne règle : la carte affiche et remonte les clics au parent via onEdit et onDelete, et c'est l'écran qui appelle l'API.

Cette logique n'est pas respectée pour FraisAddScreen-FraisEditScreen / FraisForm. FraisForm fait tout : lecture d'AsyncStorage, construction de la requête, fetch, navigation. Il faut aligner sa logique sur FraisCard.

Responsabilité	Avant	Après
Champs du formulaire, état loading, message d'erreur	FraisForm	FraisForm
Savoir si on ajoute ou on modifie	FraisForm (if (frais))	L'écran (Add ou Edit)
Récupérer user et token	AsyncStorage, dans 3 fichiers	useAuth(), dans les écrans
Construire l'URL et faire le fetch	FraisForm, Dashboard, FraisEdit	services/fraisService.js
Naviguer après l'enregistrement	FraisForm (useNavigation)	L'écran (prop navigation)

Pourquoi faire ces changements ?

- Le formulaire devient réutilisable : il ne sait pas où vont les données, il les donne à onSubmit.
- Chaque écran a un seul rôle, donc plus de if (frais) { ... } else { ... }.
- Les URL de l'API et la gestion des erreurs HTTP sont écrites une seule fois.
- Le token a une seule source, le contexte AuthContext, déjà fait pour ça.

Étape 1 : Créer services/fraisService.js

On regroupe tous les appels à l'API frais dans un fichier, comme authService.js le fait déjà pour la connexion. Créez services/fraisService.js avec le code suivant et complétez le code :

```
import { API_URL } from "../config.js";

// Fonction commune à tous les appels de l'API frais :
// ajoute le token, lit la réponse JSON et transforme les erreurs HTTP en
exceptions
async function request(endpoint, token, options = {}) {
  const response = await fetch(`${API_URL}${endpoint}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
  });

  // Certaines réponses n'ont pas de corps JSON : on renvoie null au lieu de
planter
  const data = await response.json().catch(() => null);

  if (!response.ok) {
    throw new Error(data?.error || `Erreur HTTP ${response.status}`);
  }
  return data;
}

export function getFraisList(idVisiteur, token) {
  return request(`frais/liste/${idVisiteur}`, token);
}

export function getFrais(idFrais, token) {
  // Todo : compléter en renvoyant l'appel à la fonction request() avec les bons
arguments
}

export function addFrais(frais, token) {
  return request("frais/ajout", token, {
    method: "POST",
    body: JSON.stringify(frais),
  });
}

export function updateFrais(frais, token) {
  // Todo : compléter en renvoyant l'appel à la fonction request() avec les bons
arguments
}

export function deleteFrais(idFrais, token) {
  // Todo : compléter en renvoyant l'appel à la fonction request() avec les bons
arguments
}
```

Explications du code

• La fonction request() contient le code qui était recopié partout : l'en-tête Authorization, la lecture du JSON et le contrôle de response.ok.

Cette fonction prend trois paramètres en entrée : `endpoint, token, options = {}`

- endpoint correspond à l'url de l'API
- le token est fourni par l'écran qui appelle useAuth().
- options contient ce qui change d'un appel à l'autre : la méthode HTTP (method GET, POST, PUT, DELETE) et les données envoyées (body, un objet JSON). L'opérateur de décomposition (...) ajoute tous les couples clé/valeur de options au dictionnaire qui contient déjà headers.

• Quand l'API répond une erreur, request lance une exception (throw). L'écran n'a donc qu'à utiliser try / catch, et le message de l'API (data.error) arrive jusqu'à l'utilisateur.

Étape 2 : FraisForm remonte les valeurs via onSubmit

Le formulaire reçoit une nouvelle prop, onSubmit, une fonction fournie par l'écran. Au clic sur le bouton, il l'appelle avec les valeurs saisies et affiche l'erreur si elle échoue. Il n'importe plus ni AsyncStorage, ni API_URL.

1. La signature et les états

Remplacez le début du composant (de export default function jusqu'à la fin du useEffect) par :

```
export default function FraisForm({ frais = null, onSubmit }) {
  // Utilisé uniquement pour les boutons frais hors forfait / au forfait (pas
encore implémentés)
  const navigation = useNavigation();

  // États locaux du formulaire
  const [anneeMois, setAnneeMois] = useState("");
  const [nbJustificatifs, setNbJustificatifs] = useState("");
  const [montant, setMontant] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  // En mode modification, on préremplit les champs avec le frais reçu
  useEffect(() => {
    if (frais) {
      setAnneeMois(frais.anneemois || "");
      setNbJustificatifs(String(frais.nbjustificatifs || ""));
      // Plus tard, on calculera ici le montant saisi à partir des montants des
différents frais
    }
  }, [frais]);
```

- Le useEffect préremplit les champs quand on reçoit un frais (mode modification). En mode ajout, frais vaut null et les champs restent vides.
- idFrais disparaît : c'est l'écran de modification qui connaît l'id, le formulaire n'en a plus besoin.

2. La soumission

Remplacez la fonction handleSubmit par :

```
const handleSubmit = async () => {
  setLoading(true);
  setError("");
  try {
    await onSubmit({
      anneemois: anneeMois,
      nbjustificatifs: parseInt(nbJustificatifs, 10),
    });
  } catch (err) {
    console.error("Erreur:", err);
    setError(err.message || "Erreur lors de l'enregistrement");
  } finally {
    setLoading(false);
  }
};
```

Le formulaire envoie seulement ce que l'utilisateur a saisi. id_visiteur, id_frais ou id_etat sont ajoutés par l'écran. Comme onSubmit est async, une erreur lancée par le service remonte jusqu'ici et s'affiche sous le bouton.

3. Les imports

Supprimez AsyncStorage et API_URL, vous devez avoir uniquement les imports suivants :

```
import { useState, useEffect } from "react";
import { View, Text, TextInput, Pressable } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { fraisFormStyles as styles } from "../styles/fraisFormStyles.js";
```

> **À vous de réfléchir**
>
> 1. Pourquoi FraisForm n'a-t-il plus besoin de savoir s'il s'agit d'un ajout ou d'une modification ?
> 2. Que se passerait-il si handleSubmit n'avait pas de try / catch et que le service échouait ?

Étape 3 : FraisAddScreen ajoute le frais

On ajoute l'appel au service dans l'écran d'ajout : il fournit à FraisForm une fonction handleAdd qui complète les valeurs avec l'id du visiteur, appelle le service, puis revient au tableau de bord.

```
import { View, StyleSheet } from "react-native";
import FraisForm from "../components/FraisForm";
import Navbar from "../components/Navbar";
import { useAuth } from "../context/AuthContext";
import { addFrais } from "../services/fraisService";

export default function FraisAddScreen({ navigation }) {
  const { user, token } = useAuth();

  // Appelée par FraisForm avec les valeurs saisies.
  // En cas d'erreur, l'exception remonte au formulaire qui l'affiche.
  async function handleAdd(values) {
    await addFrais({ ...values, id_visiteur: user.id_visiteur }, token);
    navigation.navigate("Dashboard");
  }

  return (
    <View style={{ flex: 1 }}>
      <Navbar />
      <View style={styles.screen}>
        <FraisForm onSubmit={handleAdd} />
      </View>
    </View>
  );
}
```

- { navigation } : un écran déclaré dans le Stack.Navigator reçoit cette prop automatiquement, donc pas besoin de useNavigation().
- { ...values, id_visiteur: ... } copie les champs saisis et ajoute l'id. Le body envoyé est identique : anneemois, nbjustificatifs, id_visiteur.
- Il n'y a pas de try / catch ici, et c'est voulu. Si addFrais échoue, l'exception sort de handleAdd et c'est le catch de FraisForm qui affiche le message. Si on l'attrapait ici, le formulaire ne verrait jamais l'erreur.
- navigation.navigate("Dashboard") n'est atteint que si l'ajout a réussi.

Étape 4 : FraisEditScreen charge et modifie le frais

L'écran de modification fait deux choses : il charge le frais avec getFrais, puis il fournit handleUpdate au formulaire.

```
import { useState, useEffect } from "react";
import { View, Text, ActivityIndicator, StyleSheet } from "react-native";
import FraisForm from "../components/FraisForm";
import Navbar from "../components/Navbar";
import { useAuth } from "../context/AuthContext";
import { getFrais, updateFrais } from "../services/fraisService";

export default function FraisEditScreen({ navigation, route }) {
  const { id } = route.params;
  const { token } = useAuth();
  const [frais, setFrais] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // Récupération du frais à modifier
  useEffect(() => {
    const fetchFrais = async () => {
      try {
        setFrais(await getFrais(id, token));
      } catch (err) {
        console.error("Erreur:", err);
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
    fetchFrais();
  }, [id, token]);

  // Appelée par FraisForm avec les valeurs saisies : on les fusionne
  // avec les champs non modifiables du frais d'origine
  async function handleUpdate(values) {
    await updateFrais(
      {
        id_frais: frais.id_frais,
        id_visiteur: frais.id_visiteur,
        id_etat: frais.id_etat, // Conserver l'état actuel
        montantvalide: frais.montantvalide,
        ...values,
      },
      token
    );
    navigation.navigate("Dashboard");
  }

  let content;
  if (loading) {
    content = <ActivityIndicator size="large" />;
  } else if (!frais) {
    content = <Text>{error || "Frais non trouvé"}</Text>;
  } else {
    content = <FraisForm frais={frais} onSubmit={handleUpdate} />;
  }

  return (
    <View style={{ flex: 1 }}>
      <Navbar />
      <View style={styles.screen}>{content}</View>
    </View>
  );
}
```

- route arrive en prop puisqu'on navigue vers l'écran FraisEdit depuis le DashboardScreen en cliquant sur le bouton modifier qui appelle la fonction handleEdit() de DashboardScreen qui exécute navigation.navigate("FraisEdit", { id });

  Donc useRoute() n'est plus nécessaire.
- Le token vient de useAuth() et non plus d'AsyncStorage. Il fait partie des dépendances du useEffect ([id, token]) puisqu'on l'utilise dedans.
- Dans handleUpdate, l'ordre compte : on écrit d'abord les champs d'origine, puis ...values. Les valeurs saisies (anneemois, nbjustificatifs) écrasent donc les anciennes, et le reste est conservé. On liste les champs un par un plutôt que ...frais pour ne pas envoyer à l'API des champs qu'elle n'attend pas, comme lib_etat.
- La variable content permet de garder la Navbar affichée dans les trois cas (chargement, erreur, formulaire). Si le chargement échoue, le vrai message d'erreur s'affiche.

Étape 5 : DashboardScreen utilise le service

Le tableau de bord garde sa logique (chargement, filtres, confirmation de suppression). On remplace seulement les deux fetch par le service et on affiche les erreurs de chargement. Effectuez les modifications suivantes dans DashboardScreen.

1. Les imports

Supprimez AsyncStorage et API_URL, et ajoutez le service :

```
import { useState, useCallback } from "react";
import { useFocusEffect } from "@react-navigation/native";
import {
  View,
  Text,
  FlatList,
  TextInput,
  Switch,
  ActivityIndicator,
  StyleSheet,
  Alert,
  Platform
} from "react-native";
import FraisCard from "../components/FraisCard";
import Navbar from "../components/Navbar";
import { useAuth } from "../context/AuthContext";
import { getFraisList, deleteFrais } from "../services/fraisService";
```

2. Un état pour l'erreur

Ajoutez, sous la déclaration de minMontant :

```
const [error, setError] = useState("");
```

3. Le chargement de la liste

Dans fetchFrais, remplacez tout le bloc try / catch par :

```
try {
  setFraisList(await getFraisList(user.id_visiteur, token));
  setError("");
} catch (err) {
  console.error("Erreur lors de la récupération des frais :", err);
  setError(err.message);
} finally {
  setLoading(false);
}
```

finally s'exécute dans tous les cas, donc un seul setLoading(false) remplace les deux d'avant.

4. La suppression

La fonction locale s'appelait deleteFrais, le même nom que la fonction importée. On la renomme removeFrais :

```
async function removeFrais(id) {
  try {
    await deleteFrais(id, token);
    setFraisList((prev) => prev.filter((f) => f.id_frais !== id));
  } catch (error) {
    Alert.alert("Erreur lors de la suppression :", error.message);
  }
}
```

> **N'oubliez pas les appels**
> Renommez aussi les deux appels dans handleDelete :
> `if (window.confirm(message)) removeFrais(id)`
> et
> `onPress: () => removeFrais(id)`

5. Afficher l'erreur

Juste avant la balise FlatList :

```
{error ? <Text style={styles.error}>{error}</Text> : null}
```

Et dans le StyleSheet :

```
error: {
  color: "red",
  marginBottom: 12,
},
```

Avant, si l'API échouait, la liste restait vide sans explication. Maintenant, on affiche une erreur en rouge.

Vérifier que tout marche

Une recherche de fetch() et de AsyncStorage dans le projet ne doit plus rien trouver en dehors de services/ et context/AuthContext.jsx. Ensuite, lancez l'application, testez et corrigez si nécessaire :

- ☐ Le tableau de bord affiche la liste des frais.
- ☐ Ajouter un frais ramène au tableau de bord, et le nouveau frais apparaît.
- ☐ Si l'API refuse un ajout ou une modification, son message s'affiche sous le bouton au lieu de rester dans la console. Testez en saisissant une lettre au lieu d'un nombre pour le nombre de justificatifs.
- ☐ « Modifier » préremplit le formulaire.
- ☐ Modifier le nombre de justificatifs puis « Mettre à jour » : la carte affiche la nouvelle valeur.
- ☐ Supprimer un frais après confirmation le retire de la liste.
- ☐ L'écran de modification affiche bien la barre de navigation.